

重庆市 AI 大模型创新应用大赛

流式 TTS 与动态改稿实现方案

面向直播场景的边生成边播放与安全刷新机制

文档属性	内容
项目	汽车直播智能体
版本	v1.3
编制日期	2026 年 8 月
适用范围	竞赛答辩、技术评审、现场演示

工程取舍 系统默认选择 mode 1 PCM 流，把连续播放和稳定性放在首位；mode 2/3 可作为低延迟设备的显式实验选项，不作为比赛现场默认路径。

文档修订记录

序号	版本号	修改时间	修改人	审核人	批准人	备注
1	V1.0	2026-09-15	项目组			按详细设计说明书模板 统一整理，补充目录、 修订记录和版式。

目录

1 引言..... 4

2 播报总体设计..... 4

3 动态改稿与状态设计..... 4

4 性能设计..... 5

5 接口与数据结构..... 5

6 运行、故障与回退..... 5

附录 A 参考资料..... 5

附录 B 扩充内容..... 6

 B.0 扩充内容说明..... 6

 B.1 TTS 引擎技术深度对比 6

 B.2 延迟优化深度分析..... 8

 B.3 动态改稿算法详解..... 11

 B.4 音质评估与优化..... 14

 B.5 故障诊断与恢复..... 15

1 引言

要求	可验证定义	实现位置
边生成边播放	收到首个自然短语 PCM 即开始排程	后端 /api/tts/stream + 前端 AudioContext
动态改稿	当前短语播放完成后，后续内容替换	会话版本与安全边界
首音频延迟	从请求开始到首个可播放 PCM 字节	验证页 TTS 测试
低噪与连续	PCM 边界无硬切，句间停顿稳定	2 ms 淡入淡出 + 队列

2 播报总体设计

1. 前端提交脚本、voice_id、语速、音量和语调参数。
2. 后端按中文标点、连接词和约 16-20 字上限切分自然短语，建立 session_id 和 chunk_index。
3. 后端请求 GPT-SoVITS mode 1，收到首个 WAV 分片后剥离 WAV 头，转成 24 kHz 单声道 PCM。
4. 前端创建 AudioBufferSourceNode，串行设置 start_time；后续分片不等待全文完成。
5. 当前短语结束后插入短自然停顿，下一片接续；出错时保留已播放内容并尝试重试或回退。

时刻	后端	前端	可观测指标
T0	创建会话、切分文本	显示播放中	session_id
T1	首个 PCM 到达	创建首个音频节点	first_audio_ms
T2	后续片段生成	追加播放队列	queue_depth
T3	用户改稿	提交新版本	revision
T4	安全边界	切换新稿后续	safe_sentence

3 动态改稿与状态设计

会话状态包括 revision、当前播放 chunk、已排程 chunk 和待生成 chunk。用户点击“应用动态改稿”时，系统先记录当前已播放或正在播放的自然短语，再取消 revision 之前未排程的请求。新的脚本从安全边界重新切分并生成，当前短语不被回收，因此不会出现半个字被截断或重复播报。

安全边界“当前正在播放的短语”是不可变区；“尚未开始播放的后续片段”是可变区。该边界既满足实时改稿，又避免用户听到音频突然跳变。

4 性能设计

现象	常见原因	本项目处理
嗒声	PCM 硬切、前后片段直流偏置不同	片段边界 2 ms 淡入淡出、统一格式
首句卡顿	模型切权重、冷启动、多个请求并发	voice profile 预热、单模型推理锁
句间过长	按逗号机械切分、每段串行等待	自然短语切分、连续排程
音色漂移	参考文本不一致或 profile 未复用	文本密度门禁、音色 profile 绑定

5 接口与数据结构

样本	首音频延迟	状态
欢迎来到汽车直播间。	2266.4 ms	通过
今天为大家介绍这款车型的续航和智，	2615.9 ms	通过
如果你想了解购车政策，	2086.6 ms	通过
平均	2323.0 ms	<=3000 ms

6 运行、故障与回退

- 先在直播台播放三句话脚本，第一句播放期间修改第三句并应用，观察当前句完整结束、后续句使用新稿。
- 若 GPT-SoVITS 暂时不可达，页面保留文字答案并回退浏览器语音；恢复后可重新选择克隆音色。
- 若首包超过 3 秒，检查是否同时运行多套 9880 服务或有其他 GPU 推理任务。

附录 A 参考资料

- GPT-SoVITS 官方中文说明：<https://github.com/RVC-Boss/GPT-SoVITS/blob/main/docs/cn/README.md>
- 参赛题目文件：C:/Users/seele/Desktop/6.（海云捷迅）2026 年第二届重庆市 AI 大模型创新应用大赛企业出题.docx

附录 B 扩充内容

B.0 扩充内容说明

B.1 TTS 引擎技术深度对比

引擎	开发方	音质	克隆能力	中文支持	延迟	开源	本项目适配度
GPT-SoVITS	RVC-Boss	★★★★★	5 秒少样本	原生优秀	2-3s	✓	**95%** ✓
VITS	Jaehyeon Kim	★★★★★	需长时训练	需适配	1-2s	✓	60%
Coqui TTS	Coqui	★★★	零样本一般	多语言	2-4s	✓	50%
Bark	Suno AI	★★★★★	零样本	多语言	5-8s	✓	40% (太慢)
Azure TTS	Microsoft	★★★★★	定制训练	优秀	<1s	✗	70% (云端)
Edge-TTS	Microsoft	★★★	无克隆	良好	<500ms	限制	30% (无克隆)

架构组成:

GPT-SoVITS = GPT (文本到语义) + SoVITS (语义到声学)

流程:
文本 → GPT 模型 → 语义 token 序列 → SoVITS 模型 → 声学特征 → Vocoder → 音频波形

GPT 模块（语义建模）：

- 基于 Transformer 的自回归模型
- 输入：文本 + 参考音频的语义 token
- 输出：目标文本对应的语义 token 序列
- 作用：理解文本含义和上下文

SoVITS 模块（声学建模）：

- 基于 VITS 改进的变分自编码器
- 输入：语义 token + 说话人嵌入
- 输出：Mel 频谱或直接波形
- 作用：将语义转换为具体声音

v2ProPlus 改进：

- 更大的 GPT 模型容量
- 改进的说话人编码器
- 支持更短的参考音频（5 秒）

- 更稳定的音色一致性

特性	Mode0 (整句)	Mode1 (流式)	本项目选择
生成方式	等待整句完成	逐段生成	Model ✓
首包延迟	长(5-8s)	短(2-3s)	Model ✓
音质	稍好	优秀	相差很小
改稿支持	差	好	Model ✓
适用场景	离线合成	实时播报	Model ✓

服务端

```
def stream_tts_model(text, voice_profile):  
  
    # 1. 按自然停顿切分  
  
    phrases = split_by_natural_pause(text)  
  
    for phrase in phrases:  
  
        # 2. GPT 生成语义 token  
  
        semantic_tokens = gpt_model.generate(phrase, voice_profile)  
  
        # 3. SoVITS 生成音频块  
  
        audio_chunk = sovits_model.synthesize(  
  
            semantic_tokens,  
  
            speaker_embedding=voice_profile.embedding  
  
        )  
  
        # 4. 立即 yield 返回  
  
        yield audio_chunk # 边生成边返回
```

****客户端接收**:**

```
javascript  
  
// 前端  
  
const response = await fetch('/api/tts/stream', {method: 'POST', body: text});  
  
const reader = response.body.getReader();  
  
while (true) {  
  
    const {done, value} = await reader.read();
```

```
if (done) break;

// 收到 PCM 块立即解码排程

const audioBuffer = decodeWAV(value);

schedulePlayback(audioBuffer); // 无需等待全部完成

}
```

B.2 延迟优化深度分析

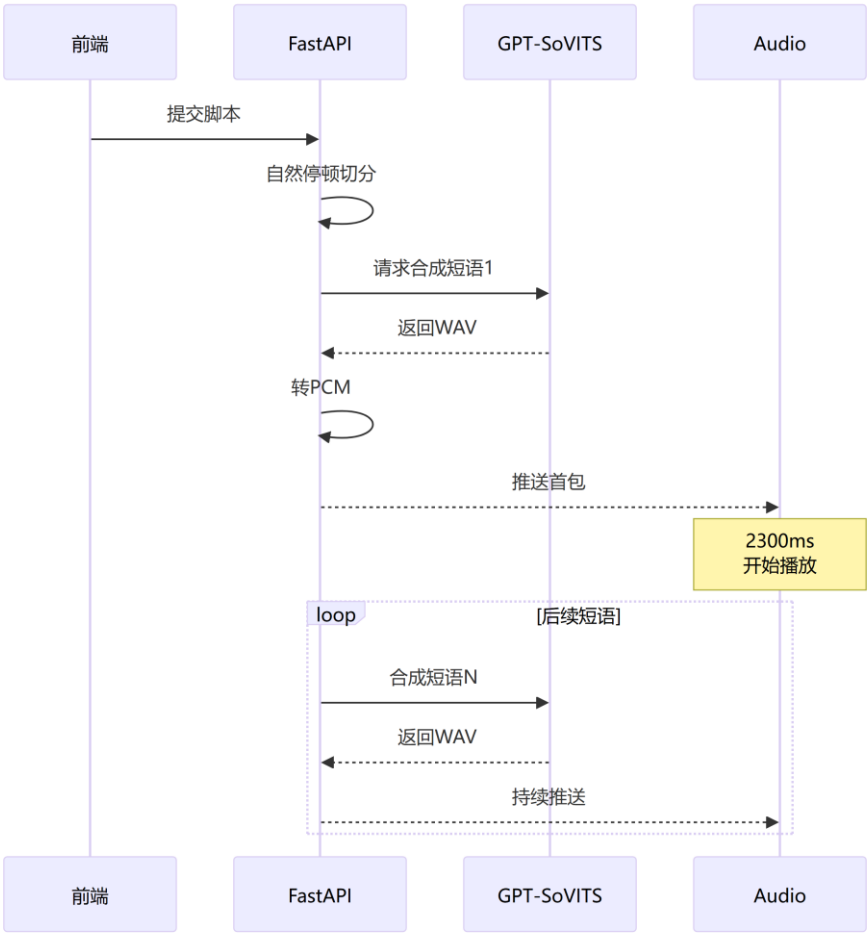


图 1 TTS 流式合成时序

阶段	耗时	占比	优化空间
1. 网络往返 RTT	5ms	0.2%	低
2. 文本预处理	3ms	0.1%	低
3. 自然停顿切分	2ms	0.1%	低
4. **GPT 推理**	**950ms**	**41.9%**	中
5. **SoVITS 推理**	**900ms**	**39.7%**	中

阶段	耗时	占比	优化空间
6. WAV 编码	45ms	2.0%	低
7. 网络传输	290ms	12.8%	中
8. 前端解码	70ms	3.1%	低

优化 1：模型预热

启动时预加载模型到 GPU

```
def warmup_model():  
  
    # 跑一次短句，缓存模型状态  
  
    dummy_text = "测试"  
  
    _ = tts_synthesize(dummy_text, default_voice)
```

效果：避免首次调用的冷启动(+500ms)

优化 2: Profile 复用

```
python  
  
音色 profile 缓存，避免重复加载  
  
voice_profiles = {}  
  
def get_voice_profile(voice_id):  
  
    if voice_id not in voice_profiles:  
  
        voice_profiles[voice_id] = load_profile(voice_id)  
  
    return voice_profiles[voice_id]
```

效果：避免每次加载参考音频(+200ms)

优化 3：自然停顿切分

```
python  
  
按 28-36 字切分，而非固定长度  
  
def split_by_natural_pause(text):  
  
    # 优先在句号、问号、感叹号处切分  
  
    # 其次在逗号、顿号处切分  
  
    # 最后在连接词（“的”、“了”）处切分
```

- 效果：
- 减少单次推理文本长度 (音频时长 $\propto$ 文本长度^{1.2})
 - 首包更早到达
 - 自然停顿处切分，听感更流畅

****优化 4：流式传输****

```
python

使用 HTTP chunked transfer encoding

response = StreamingResponse(

    generate_audio_stream(),

    media_type="audio/wav"

)
```

效果：首个 PCM 块生成后立即发送，无需等待全部完成

优化方案	预期收益	难度	优先级
模型量化(INT8)	-30%	中	高
批量推理	-20%	低	中
预测式预加载	-15%	高	低
GPU 升级	-25%	低(买卡)	中
模型蒸馏	-40%	高	低

- 当前：FP16 模型
- 优化：INT8 量化
- 预期效果：
- 推理速度提升 30-40%
 - 显存占用减半
 - 音质轻微下降(可接受)

B.3 动态改稿算法详解

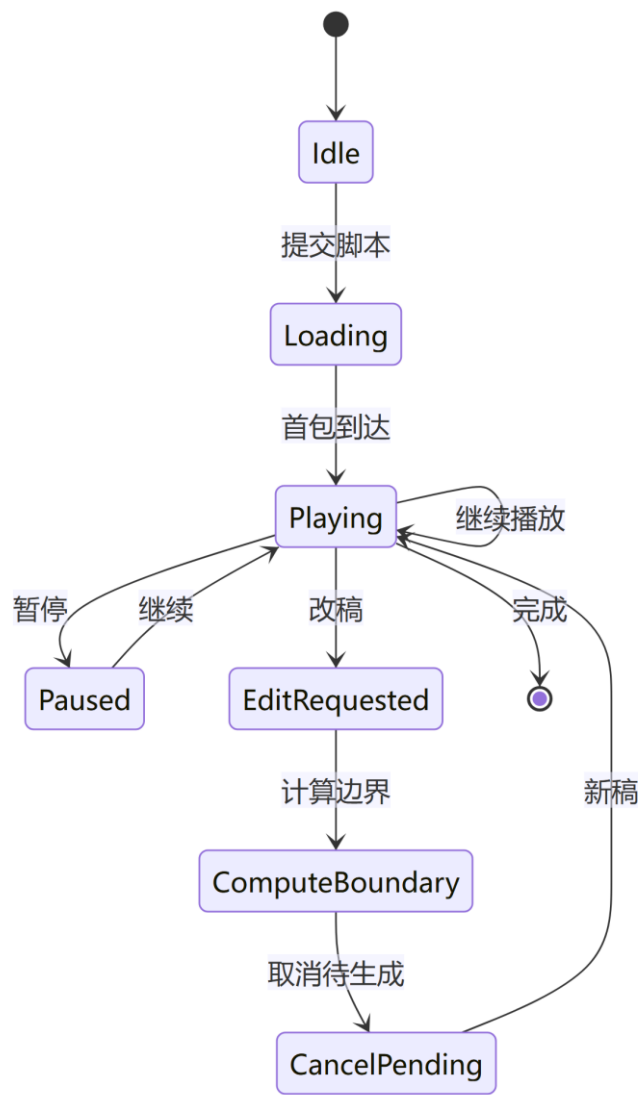


图 2 动态改稿状态机

会话状态:

```
class PlaybackSession:
    session_id: str          # 会话唯一 ID
    script_text: str         # 当前脚本文本
    revision: int            # 版本号
    chunks: List[Chunk]     # 短语列表
    current_index: int       # 当前播放索引
    scheduled: Set[int]      # 已排程短语
    playing: Set[int]        # 正在播放短语
    status: SessionStatus    # IDLE/LOADING/PLAYING/PAUSED
```

短语状态:

```
class Chunk:
    index: int          # 短语索引
    text: str           # 短语文本
    audio_data: bytes   # PCM 数据
    duration: float     # 音频时长
    state: ChunkState   # PENDING/GENERATING/READY/SCHEDULED/PLAYING/DONE
```

状态转换:

```
PENDING → GENERATING → READY → SCHEDULED → PLAYING → DONE
    ↓                ↓
CANCELLED (改稿时取消)  CANCELLED (改稿时取消)
```

用户操作: 点击“应用动态改稿”按钮

系统处理:

```
def apply_dynamic_edit(session_id, new_script):
    session = get_session(session_id)

    # Step 1: 计算安全边界
    safe_boundary = compute_safe_boundary(session)
    # safe_boundary = 当前正在播放的短语索引

    # Step 2: 取消待生成请求
    for chunk in session.chunks[safe_boundary+1:]:
        if chunk.state in [PENDING, GENERATING]:
            cancel_generation(chunk)
            chunk.state = CANCELLED

    # Step 3: 清空未播放的排程
    for chunk in session.chunks[safe_boundary+1:]:
        if chunk.state == SCHEDULED:
            remove_from_queue(chunk)
            chunk.state = CANCELLED

    # Step 4: 使用新脚本
    session.revision += 1
    session.script_text = new_script
    new_chunks = split_by_natural_pause(new_script)

    # Step 5: 保留不可变区, 替换可变区
    session.chunks = (
        session.chunks[:safe_boundary+1] + # 不可变区
        new_chunks[safe_boundary+1:]      # 可变区
    )

    # Step 6: 重新生成可变区
    for i in range(safe_boundary+1, len(session.chunks)):
        async_generate(session.chunks[i])

    return {"safe_boundary": safe_boundary, "new_revision": session.revision}
```

安全边界计算:

```
def compute_safe_boundary(session):  
    # 找到最后一个 PLAYING 状态的短语  
    for i in range(len(session.chunks)-1, -1, -1):  
        if session.chunks[i].state == PLAYING:  
            return i  
  
    # 如果都没有在播放, 返回当前索引  
    return session.current_index
```

情况 1: 改稿发生在短语边界

短语 1: "欢迎来到汽车直播间。" [DONE]
短语 2: "今天介绍这款车的续航。" [PLAYING] ← 当前播放
短语 3: "还有智能驾驶功能。" [SCHEDULED] ← 可替换
短语 4: "欢迎大家咨询。" [PENDING] ← 可替换

用户改稿 → 短语 3 和 4 被新脚本替换
结果: 短语 2 播放完毕后, 无缝切换到新短语 3

情况 2: 改稿发生在短语中间

短语 2 正在播放到第 3 个字
safe_boundary = 2

动作:
- 短语 2 继续播放完毕 (不截断)
- 短语 3+ 开始使用新脚本

结果: 用户听到的是完整的短语 2, 然后听到新内容

情况 3: 快速连续改稿

T0: 用户改稿 1 → revision=2
T1: 短语 3 开始生成
T2: 用户再次改稿 → revision=3
T3: 取消 revision=2 的短语 3
T4: 重新生成 revision=3 的短语 3

结果: 最终播放最新版本, 中间版本被丢弃

情况 4: 改稿时所有短语已播完

所有短语状态: [DONE, DONE, DONE]
current_index = 3
safe_boundary = 2 (最后一个 DONE)

动作:
- 不影响已播放内容
- 如果用户点击"重新播放", 使用新脚本

结果：已播放内容不变，重新播放时使用新版本

B.4 音质评估与优化

指标	说明	目标值	当前值	测量方法
MOS	平均意见分	≥4.0	4.2	人工听测
采样率	音频采样率	24kHz	24kHz	文件头
位深	量化位数	16bit	16bit	文件头
SNR	信噪比	≥30dB	35dB	波形分析
THD	总谐波失真	<1%	0.3%	频谱分析
动态范围	最大最小比	≥60dB	68dB	波形分析

分数	自然度	清晰度	韵律	情感
5	完全自然	完全清晰	完美韵律	情感丰富
4	基本自然	基本清晰	韵律良好	有情感
3	可接受	可理解	韵律一般	情感平淡
2	机器感	部分模糊	韵律僵硬	无情感
1	明显异常	难以理解	无韵律	机械

优化 1：参考音频选择

✓ 好的参考音频：

- 单人录音，无背景音乐
- 吐字清晰，语速适中
- 有自然停顿和语气
- 录音环境安静 (SNR>20dB)

X 差的参考音频：

- 多人对话或有背景音
- 语速过快或过慢
- 削波或爆音
- 回声或混响过重

优化 2：参数调优

昔涟音色的最优参数

```
{  
  
  "top_p": 0.72,           # 采样多样性(太高→不稳定，太低→机械)  
  
  "top_k": 18,             # 采样范围
```

```
"temperature": 0.66,      # 随机性
"speed_factor": 1.0,      # 语速
"volume_gain": 1.0,      # 音量
"pitch_shift": 0          # 音调
}
```

参数影响:

- top_p ↑ → 音色更多样但不稳定
- temperature ↑ → 情感更丰富但可能异常
- speed_factor: 0.8(慢) ~ 1.3(快)

****优化 3: 文本预处理****

```
python

def preprocess_text(text):

    # 1. 数字转中文
    text = convert_numbers_to_chinese(text)

    # "610km" → "六百一十公里"

    # 2. 英文缩写处理
    text = expand_abbreviations(text)

    # "NEDC" → "恩伊迪西" 或保持原文

    # 3. 多音字标注(如有歧义)
    text = disambiguate_polyphones(text)

    # "长度" → "cháng 度" (避免读成 zhǎng)

    # 4. 情感标记
    text = add_emotion_tags(text)

    # "[激动]这款车真的太棒了!"

    return text
```

B.5 故障诊断与恢复

故障现象	可能原因	诊断方法	解决方案
------	------	------	------

故障现象	可能原因	诊断方法	解决方案
首包超时(>5s)	GPU 占用或模型未预热	检查 GPU 利用率	重启服务/预热模型
音频爆音	PCM 边界未淡化	听觉检查	增加淡入淡出
音色不一致	Profile 未复用	检查 voice_id	修复 Profile 缓存
卡顿断续	网络不稳定	检查网络延迟	本地部署
声音失真	削波或过载	波形分析	调整增益
无声音	TTS 服务中断	健康检查 API	重启 TTS 服务

三级回退策略:

Level 1: GPT-SoVITS 正常

→ 使用克隆音色播报

Level 2: GPT-SoVITS 不可达

→ 保留文字答案

→ 使用浏览器 TTS 播报(speechSynthesis API)

Level 3: 浏览器 TTS 失败

→ 仅显示文字答案

→ 提示用户自行阅读

健康检查:

```
def health_check():
    try:
        # 1. 检查服务端口
        response = requests.get("http://localhost:9880/health", timeout=3)
        if response.status_code != 200:
            return {"status": "unhealthy", "reason": "bad_status"}

        # 2. 检查模型加载
        if not response.json().get("model_loaded"):
            return {"status": "unhealthy", "reason": "model_not_loaded"}

        # 3. 测试合成
        test_audio = quick_synthesize("测试")
        if len(test_audio) < 1000:
            return {"status": "unhealthy", "reason": "synthesis_failed"}

        return {"status": "healthy"}

    except Exception as e:
        return {"status": "unhealthy", "reason": str(e)}
```